

INT3420E 1 - Final Project Report

Team 14

Tran Gia Khanh
23021599
23021599@vnu.edu.vn

Pham Anh Tuan
23021709
23021709@vnu.edu.vn

Nguyen Dieu Mai Vy
23021755
23021755@vnu.edu.vn

Abstract—This report presents an experimental study on fine-tuning a Vietnamese GPT-2 model for solving Vietnamese mathematical word problems. The task requires the model to generate the final numerical answer from a given problem statement, and the performance is evaluated using a score based on the relative error between the predicted and ground-truth answers. Starting from a LoRA fine-tuning baseline with full-solution generation, we explored multiple training and inference strategies, including full reasoning targets, step-format solutions, short-reasoning targets, answer-only fine-tuning, self-consistency voting, rule-based verification, and different data filtering strategies. Experimental results show that generating long reasoning chains is less effective for this scoring setting, as it often introduces arithmetic errors and unstable output formats. The best performance was obtained by training the model with an answer-only target, removing noisy answer-augmented data, and preserving rephrased, solve-variable, and reverse-parameter problem types. The final model, based on *NlpHUST/gpt2-vietnamese* with LoRA fine-tuning, achieved a validation score 10 of 1.995 with an answer extraction rate of 1.0. These results indicate that target simplification and data selection are more important than complex reasoning generation for this task.

Index Terms—Vietnamese mathematical word problems, GPT-2, LoRA fine-tuning, answer-only generation, data preprocessing, numerical reasoning.

I. Introduction

Mathematical word problem solving is an important task in natural language processing because it requires a model to understand a textual problem statement, identify the relevant quantities, and produce a final numerical answer. Solving mathematical word problems requires both natural language understanding and multi-step numerical reasoning, which remains a significant challenge for language models [1]. Small numerical mistakes can lead to completely incorrect outputs, making this task particularly sensitive to reasoning errors. This difficulty becomes more significant in Vietnamese, where available mathematical reasoning resources and pretrained models are more limited than those for English.

In this report, we study the use of a Vietnamese GPT-2 model for solving Vietnamese mathematical word problems. The input of the system is a problem statement written in natural language, and the expected output is the final answer. Although large language models can generate detailed reasoning steps, the evaluation setting of this task mainly focuses on the correctness of the final numerical answer. Therefore, the central question of this work is how to adapt a Vietnamese generative language model so that it can produce stable and

extractable final answers under a constrained model and data setting.

The task presents several practical challenges. First, the dataset contains multiple problem types, including direct-answer problems, rephrased problems, solve-variable problems, and reverse-parameter problems. These types differ not only in wording but also in the objective that the model needs to infer. Second, the training data may contain duplicated samples, noisy augmented examples, long mathematical expressions, and answers with inconsistent formats. Third, since the base model is a general-purpose Vietnamese GPT-2 model rather than a specialized mathematical solver, the fine-tuning strategy and output format have a large influence on the final performance.

To address these challenges, this report investigates a sequence of experimental variants built around LoRA fine-tuning [2]. The experiments compare different target formats, data filtering strategies, and inference settings in order to identify which design choices are most suitable for this task. Instead of treating all generated text equally, the study focuses on the relationship between the training objective, the distribution of problem types, and the ability of the model to generate a valid final numerical answer.

The main contributions of this report are summarized as follows:

- We construct a fine-tuning pipeline for adapting a Vietnamese GPT-2 model to mathematical word problem solving.
- We analyze how different output target formats affect the stability of final-answer generation.
- We examine the impact of data preprocessing and problem-type selection on validation performance.
- We compare multiple experimental variants and select a final model configuration based on empirical results.

Overall, this work aims to provide a practical experimental analysis of how a relatively small Vietnamese language model can be adapted to a numerical answer generation task through careful data selection, target design, and fine-tuning configuration.

II. Dataset Analysis and Preprocessing

A. Dataset Structure

The dataset consists of Vietnamese mathematical word problems designed for causal language model fine-tuning.

Each record contains seven fields: `original_question_vi`, `original_question_en`, `query_vi`, `query_en`, `response_vi`, `response_en`, and `type`. In this work, the Vietnamese query field, `query_vi`, is treated as the model input, while `response_vi` is used to derive the target answer.

A key characteristic of the dataset is that `query_vi` is not always identical to `original_question_vi`. Many samples are transformed versions of the original problem, such as rephrased questions, answer-augmented variants, solve-variable problems, or reverse-parameter problems. Therefore, replacing `query_vi` with `original_question_vi` would be incorrect, as it may change the objective of the problem. The `type` field is mainly used for data analysis, stratified evaluation, and error analysis rather than as a direct input prompt feature.

The training set contains 100,000 records with no missing fields or empty values. However, the dataset is highly augmented: there are 58,925 unique `query_vi` values but only 13,101 unique `original_question_vi` values. This confirms that multiple transformed questions can originate from the same base problem.

Table I
General statistics of the training data

Property	Value
Total records	100,000
Fields per record	7
Missing fields	0
Empty fields	0
Unique <code>query_vi</code>	58,925
Unique original questions	13,101
Exact duplicate query-response rows	1,032
Conflicting duplicate queries	106 queries / 447 rows
Records where <code>query_vi</code> differs from original question	60,619

B. Problem Type Distribution

The dataset contains both GSM-style word problems and MATH-style problems. GSM problems are generally closer to everyday arithmetic reasoning, while MATH problems often contain more formal mathematical expressions, LaTeX syntax, or symbolic answers. The dataset is further divided into four augmentation or transformation styles: `AnsAug`, `Rephrased`, `SV`, and `FOBAR`.

The `AnsAug` and `Rephrased` groups form the majority of the data. The `SV` and `FOBAR` groups are smaller but important because they change the objective of the problem. In solve-variable problems, the model must infer an unknown variable rather than simply answer the original question. In reverse-parameter problems, the model must infer a missing parameter that makes the given condition true.

Table II
Distribution of problem types in `TRAIN.JSON`

Type	Count	Ratio
GSM_AnsAug	20,329	20.3%
GSM_Rephrased	20,298	20.3%
MATH_AnsAug	18,991	19.0%
MATH_Rephrased	12,788	12.8%
GSM_FOVAR	10,191	10.2%
GSM_SV	9,920	9.9%
MATH_FOVAR	3,769	3.8%
MATH_SV	3,714	3.7%

When grouped by source, the dataset contains 60,738 GSM samples and 39,262 MATH samples. When grouped by transformation style, there are 39,320 `AnsAug` samples, 33,086 `Rephrased` samples, 13,960 `FOVAR` samples, and 13,634 `SV` samples. This distribution shows that the training data is not a simple collection of original question-answer pairs, but a heavily transformed dataset with multiple objectives.

C. Length Characteristics

The length distribution of the dataset is important because the model is fine-tuned as a causal language model with a fixed maximum sequence length. Most samples are short enough to fit within a moderate context window. The median length of `query_vi` is 196 characters, and the median length of `response_vi` is 407 characters. The median combined prompt-response length is 635 characters.

However, a small portion of the dataset contains very long samples, especially in the MATH subset. Some records contain long LaTeX expressions or [asy] diagram code, which can substantially increase the sequence length. If such samples are truncated naively, the final answer may be removed from the training target.

Table III
Length statistics of Vietnamese fields

Field	Median	Mean	P95	Max
<code>query_vi</code> chars	196	212.3	418	2,310
<code>response_vi</code> chars	407	475.6	974	3,444
Prompt + response chars	635	707.9	1,340	3,841
<code>query_vi</code> words	46	48.6	96	311
<code>response_vi</code> words	97	113.5	237	772
Prompt + response words	149	166.0	320	850

Approximately 16.87% of the samples have a combined prompt-response length greater than 1,000 characters, while only 0.65% exceed 2,000 characters. This suggests that most records can be used for fine-tuning with a fixed-length context, but long mathematical or diagram-heavy samples require additional filtering or careful truncation.

D. Answer Format Characteristics

Most responses end with an explicit answer anchor. The two most common Vietnamese answer patterns are `Đáp án là:` and `Câu trả lời là:`. These anchors are useful for extracting the final answer and standardizing the model target. In addition, many responses include answer markers such as `####` or LaTeX boxes such as `\boxed{}`.

The majority of final answers are plain integers, but a non-negligible portion contains fractions, decimals, symbolic expressions, radicals, or LaTeX notation. Therefore, preprocessing should not assume that every valid answer is a simple integer.

Table IV
Common final answer formats

Answer format	Count
Plain integer	91,713
LaTeX fraction, e.g., $\frac{1}{48}$	3,455
Text or symbolic expression	2,388
Decimal using comma	982
Decimal using dot	558
Base subscript, e.g., 2104_5	53
Not extracted by a simple parser	203

This observation motivates a conservative answer extraction strategy. The preprocessing stage should normalize common numeric formats but should avoid aggressively converting all mathematical expressions into decimals, because doing so may distort symbolic or LaTeX-based answers.

E. Noise and Data Quality Issues

Although the dataset is complete at the schema level, it contains several format-level and content-level issues. First, there are duplicated query-response pairs and a smaller number of conflicting duplicate queries, where the same query is associated with different answers. These conflicts can introduce contradictory supervision during fine-tuning.

Second, many samples contain mathematical syntax, decimal commas, LaTeX expressions, or translated LaTeX artifacts such as an incorrectly translated `\boxed{}` command. These patterns must be normalized carefully because they are closely related to the final answer. Third, some MATH samples include `[asy]` diagram code, which is very long and may add noise to the textual input. Finally, some samples contain English leakage, which is not ideal when the model is trained primarily on Vietnamese inputs.

Table V
Observed data quality issues

Issue	Count
query_vi differs from original question	60,619
Queries containing unknown variable x/X	35,683
Unknown-variable phrase appears in query	27,446
Repeated variable-related phrase	3,201
LaTeX or mathematical syntax	33,685
Decimal comma	17,173
<code>[asy]</code> diagram code	1,012
Incorrectly translated <code>\boxed</code> command	752
Exact duplicate query-response rows	1,032
Conflicting duplicate query rows	447

An important preprocessing decision is to avoid deduplicating records based only on `original_question_vi`. Since multiple valid transformed questions may be derived from the same original problem, deduplicating by the original

question could remove useful samples or mix different targets. Therefore, deduplication should be applied at the query-answer level, while conflicting query groups should be removed or handled separately.

F. Preprocessing Design

Based on the dataset analysis, the preprocessing pipeline is designed to preserve the actual Vietnamese input-target relationship while reducing format noise. The main preprocessing steps are as follows:

- 1) Use `query_vi` as the input field and `response_vi` as the source for answer extraction.
- 2) Remove or filter extremely long diagram-heavy content, especially `[asy]` blocks.
- 3) Normalize common formatting artifacts, including whitespace, answer anchors, decimal separators, and translated LaTeX commands.
- 4) Extract the final answer from common Vietnamese answer anchors, the `####` marker, and LaTeX boxed expressions such as `\boxed{}`.
- 5) Remove exact duplicate query-answer pairs.
- 6) Remove or filter conflicting duplicate queries where the same query maps to inconsistent answers.
- 7) Control very long samples to reduce the risk of truncating the final answer.

The purpose of preprocessing is not to over-clean the mathematical content, but to make the target format more consistent and easier to learn. In later experiments, this analysis also guides the data selection strategy, especially the decision to compare different problem-type distributions rather than treating all augmented samples equally.

III. Methodology

This section describes the methodological choices used to adapt a Vietnamese GPT-2 model to the mathematical word problem answering task. The main objective is to fine-tune a causal language model so that, given a Vietnamese problem statement, it can generate a valid and extractable final numerical answer. The methodology is designed around three main considerations: the limited reasoning capacity of the base model, the alignment between the training target and the evaluation metric, and the influence of data selection on final-answer generation.

A. Base Model and LoRA Fine-tuning

The base model used in this work is `NlpHUST/gpt2-vietnamese`, a GPT-2 style causal language model pretrained for Vietnamese text generation. Since the task requires generating the continuation of a given prompt, the model is fine-tuned under the causal language modeling framework.

Instead of updating all model parameters, Low-Rank Adaptation (LoRA) [2] is applied to make fine-tuning more efficient. LoRA introduces trainable low-rank matrices into selected projection layers while keeping the original pretrained weights frozen. This reduces the number of trainable parameters and

allows the model to be adapted under limited computational resources.

Given a training sample consisting of a Vietnamese query q and a target output y , the model is trained to maximize the conditional likelihood of the target tokens:

$$\mathcal{L} = - \sum_{t=1}^T \log P(y_t \mid q, y_{<t}) \quad (1)$$

In implementation, the prompt tokens are masked from the loss. Therefore, the model is optimized only on the target tokens, such as the final answer or the solution text, depending on the experimental variant. This prevents the model from being penalized for reproducing the input prompt and focuses the learning signal on the desired output.

B. Model Capacity and Design Rationale

An important consideration in this work is the capacity limitation of the selected base model. The required model is a GPT-2 style causal language model rather than a specialized mathematical reasoning system. Under the official task constraints, the base model cannot be replaced by a larger backbone, and the model must remain frozen during inference. Therefore, the design of the fine-tuning target and the inference strategy becomes critical.

In early experiments, the model was trained to generate full Vietnamese solutions before producing the final answer. Although this format is natural for mathematical word problems, it creates a long generation path. For a relatively small causal language model, each additional generated token increases the probability of repetition, arithmetic inconsistency, or deviation from the required answer format. Moreover, the evaluation metric depends only on the final numerical answer, not on the quality of the intermediate explanation. As a result, optimizing the model to generate long reasoning chains may not be well aligned with the scoring objective.

Another practical limitation is the maximum number of tokens generated at inference time. If `max_new_tokens` is too small, a solution-style output may be truncated before the final answer appears. If it is too large, the model has more opportunity to produce verbose, repetitive, or unstable text. This trade-off was observed in several solution-based and short-reasoning variants. Even when the final answer is eventually generated, errors in the intermediate reasoning often lead to an incorrect numerical result.

Based on these observations, the experimental direction gradually shifted from solution generation to answer-only generation. The answer-only target reduces the output space, shortens the decoding process, and aligns the training objective more directly with the evaluation metric. Instead of forcing the model to imitate a full reasoning trace, the model is trained to map a Vietnamese problem statement to its final answer in a consistent and extractable format. This design is especially suitable for the constrained model setting, where answer stability is more important than generating detailed explanations.

C. Prompt Formulation

The input prompt is designed to be simple and consistent across samples. The main prompt format used in the answer-only setting is:

Câu hỏi: <query_vi>
Đáp án:

The model is then trained to generate the final answer following this prompt. This format is intentionally minimal. It avoids adding task labels, English instructions, or additional metadata because the input should primarily rely on the Vietnamese problem statement. The `type` field is used for data analysis, stratified evaluation, and error analysis, but it is not included in the final prompt.

D. Target Format Design

Based on the model-capacity considerations discussed above, several target formats were explored to determine whether the model should generate a full solution, a structured explanation, a shortened reasoning trace, or only the final answer. Since the task is evaluated based on the final numerical answer, the target format has a strong influence on model behavior.

1) *Full-solution Target*: Generating a step-by-step reasoning path, commonly known as Chain-of-Thought (CoT), has been shown to elicit reasoning capabilities in large language models [3]. The initial approach trains the model to generate a full solution followed by the final answer. The target output keeps most of the original `response_vi` content and standardizes the final answer line when possible:

Lời giải: ...
Đáp án là: <answer>

This format allows the model to learn intermediate reasoning patterns. However, it also increases output length and may introduce arithmetic or formatting errors before the final answer is produced.

2) *Step-format Target*: Another explored format is step-based reasoning, where the original solution is converted into ordered steps:

Bước 1: ...
Bước 2: ...
Đáp án là: <answer>

This format is useful for making the generated explanation more structured. Nevertheless, because the evaluation focuses only on the final answer, step-format generation may allocate too much modeling capacity to explanation tokens instead of answer tokens.

3) *Short-reasoning Target*: The short-reasoning target keeps only a compact explanation before the answer:

Lời giải ngắn: ...
Đáp án là: <answer>

This design attempts to balance reasoning and answer generation. It reduces the output length compared with full-solution

generation, but the model still needs to generate intermediate text before producing the final answer.

4) *Answer-only Target*: The final selected target design is answer-only generation. Prior research has shown distinct trade-offs between process-based and outcome-based supervision [4]. In our constrained GPT-2 setting, enforcing complex reasoning paths often increases the risk of arithmetic inconsistencies and unstable output formats. Therefore, answer-only generation is used as a more direct way to align the training objective with the evaluation metric.

In this setting, the model is trained to generate only the final answer or a short final-answer statement:

<answer>

or

Đáp án là: <answer>

This target directly matches the evaluation objective. It reduces unnecessary generation, improves output consistency, and lowers the chance of producing invalid or repetitive reasoning text.

Table VI
Target format variants explored in the experiments

Target format	Description
Full solution	Generate a complete solution followed by the final answer.
Step-format	Convert the solution into ordered reasoning steps.
Short reasoning	Generate a concise explanation before the answer.
Answer-only	Generate only the final answer or final answer statement.

E. Answer Extraction and Target Standardization

Since the original responses contain multiple answer styles, a target standardization step is required before fine-tuning. The final answer is extracted from common answer markers such as:

- Đáp án là:
- Câu trả lời là:
- ####
- \boxed{}

After extraction, the answer is normalized into a consistent target format. For answer-only training, the target is reduced to the final extracted answer. For solution-based variants, the final answer line is standardized to reduce output ambiguity.

This step is important because inconsistent answer anchors can make the model learn multiple incompatible output patterns. A consistent target format increases the probability that the generated answer can be extracted during evaluation.

F. Data Selection Strategy

The methodology also includes several data selection strategies. Early experiments considered using all available problem types or selecting examples based on general quality filtering.

Later experiments showed that the distribution of problem types has a strong influence on performance.

The dataset contains several transformed problem types, including answer-augmented, rephrased, solve-variable, and reverse-parameter samples. In the final configuration, the answer-augmented groups are removed from the training subset, while the rephrased, solve-variable, and reverse-parameter groups are preserved. The final training subset is selected from:

- GSM_Rephrased
- MATH_Rephrased
- GSM_SV
- MATH_SV
- GSM_FOBAR
- MATH_FOBAR

The two excluded groups are:

- GSM_AnsAug
- MATH_AnsAug

This design is based on the observation that not all augmented data contributes equally to final answer generation. In particular, preserving solve-variable and reverse-parameter samples is important because these groups require the model to infer objectives that differ from direct-answer problems. At the same time, completely removing the answer-augmented groups should be interpreted as an empirical choice based on validation performance, not as a general claim that all answer-augmented examples are harmful.

G. Inference Strategies

Several inference strategies were explored to determine how the fine-tuned model should generate final predictions.

1) *Greedy Decoding*: Greedy decoding selects the most likely token at each generation step. It is simple, deterministic, and efficient. This strategy is especially suitable for answer-only generation, where the output sequence is short and does not require diverse reasoning paths.

2) *Beam Search*: Beam search keeps multiple high-probability hypotheses during generation. It was considered for solution-based outputs, but for short answer-only generation, beam search did not provide a clear methodological advantage over greedy decoding.

3) *Model-only Voting*: Model-only voting generates multiple candidate answers from the fine-tuned model and then selects the most frequent canonical answer. This strategy was used in Variant 11. The first candidate is generated using greedy decoding, while additional candidates are generated using light sampling. The final answer is selected only from model-generated candidates.

This setting is useful for evaluating whether candidate aggregation can improve answer stability without using hand-crafted rules or external solvers. However, voting does not introduce new mathematical knowledge. If the model repeatedly generates similar incorrect answers, majority voting may reinforce the error rather than correct it.

4) *Verifier-based Candidate Selection*: Verifier-based selection generates multiple candidates and ranks them using simple quality checks, such as whether the output contains a valid answer, whether it repeats excessively, or whether the answer format is extractable. This strategy is useful for analyzing output quality, but it does not fully solve mathematical reasoning errors.

5) *Rule-based Fallback*: Some experiments also considered rule-based fallback methods for highly specific patterns, such as simple arithmetic expressions, greatest-common-divisor problems, least-common-multiple problems, and simple perimeter templates. This strategy was used in Variant 7.

In Variant 7, the rule solver is applied before model generation. If a question matches a conservative rule pattern, the answer is produced by the rule solver. Otherwise, the fine-tuned model generates the answer. Therefore, Variant 7 should be interpreted as a hybrid rule-first pipeline rather than a purely model-only system.

Although rule-based fallback can improve some specific cases, it is not used in the final method. The final method avoids rule-based answer generation and uses a model-only inference setting. This keeps the final system simpler and makes the prediction pipeline more representative of the fine-tuned language model itself.

Table VII
Inference strategies considered in the experimental process

Strategy	Description
Greedy decoding	Deterministic generation using the most likely next token.
Beam search	Maintains multiple high-probability decoding paths.
Model-only voting	Generates multiple model candidates and selects the majority answer.
Verifier selection	Ranks candidates using output quality heuristics.
Rule-based fallback	Uses hand-crafted rules for specific problem patterns; used only as an analytical hybrid variant, not as the final method.

H. Final Method

Based on the above design choices, the final method uses answer-only LoRA fine-tuning and model-only greedy decoding. The training data is selected by removing the answer-augmented groups and preserving rephrased, solve-variable, and reverse-parameter problem types. The final prompt is kept minimal, and the model is trained to generate only the final answer.

The final methodology can be summarized as follows:

- 1) Use `query_vi` as the Vietnamese input problem.
- 2) Extract and normalize the final answer from `response_vi`.
- 3) Remove noisy or conflicting samples during preprocessing.
- 4) Exclude `GSM_AnsAug` and `MATH_AnsAug` from the training subset.

- 5) Fine-tune `NlpHUST/gpt2-vietnamese` using LoRA with an answer-only target.
- 6) Generate predictions using greedy decoding without rule-based solvers.

Although Variant 7 achieves a competitive validation score, it is not selected as the final method because it introduces hand-crafted rule-based answer generation during inference. This makes the pipeline less purely model-based. In contrast, Variant 11 and the final configuration use only model-generated candidates. Therefore, Variant 7 is treated as an analytical variant for understanding the potential benefit of deterministic fallback rules, while the final method prioritizes a simpler model-only inference setting.

Compared with Variant 11, the final method also removes candidate voting and uses greedy decoding. This choice is based on the observation that voting cannot reliably correct numerical errors when all candidates are generated by the same small language model. The final methodology therefore emphasizes target simplification and data selection rather than complex generation or post-processing. It is designed to make the model produce stable, extractable final answers under a constrained fine-tuning setup.

IV. Experimental Setup

This section describes the experimental setting used to train, validate, and compare different model variants. The purpose of this section is to clarify the runtime constraints, evaluation metric, fine-tuning configuration, and experimental design before presenting the validation results.

A. Task Setting and Runtime Constraints

The task requires fine-tuning the fixed base model `NlpHUST/gpt2-vietnamese` to solve Vietnamese mathematical word problems. For each input question, the model must generate an output that contains the final answer. During local development, `train.json` is used for fine-tuning and `valid.json` is used for validation and error analysis.

The official runtime setting imposes several constraints. The Kaggle notebook must be executed with Internet disabled and GPU acceleration enabled. The total runtime must remain within three hours. In addition, the base model cannot be replaced by another model, and no external LLMs, APIs, online solvers, or additional training data are allowed. During inference, the model must remain frozen, meaning that no gradient update or test-time adaptation is performed.

For token generation, the special token id is set as follows:

`SAFE_EOS_ID = 50256`

This value is used for both `pad_token_id` and `eos_token_id` to avoid token-id errors during training and generation.

B. Input and Output Format

The model input is constructed from the Vietnamese query field:

Câu hỏi: `<query_vi>`

Đáp án:

The model output is expected to contain a short final answer. In the final configuration, the model is trained using an answer-only target. During prediction, the generated answer is wrapped into a consistent output format:

Đáp án là: <answer>

This format is used because the evaluator extracts the final numerical answer from the generated text. A consistent answer anchor reduces extraction failures and makes the output easier to analyze.

C. Evaluation Metric

The official score is based on the relative error between the predicted answer and the gold answer, following evaluation strategies established in mathematical reasoning benchmarks such as GSM8K [1]. For a prediction $\hat{y}$ and a gold answer y , the relative error is computed as:

$$\text{relative_error} = \frac{|\hat{y} - y|}{\max(1, |y|)} \quad (2)$$

Each example is assigned a score according to the error threshold shown in Table VIII.

Table VIII
Per-example scoring rule

Condition	Score
relative_error $\leq$ 0.01	10
relative_error $\leq$ 0.10	5
relative_error $\leq$ 0.50	1
Not extractable or too inaccurate	0

The raw score is the sum of all per-example scores. If the validation or test set contains N examples, the normalized score is:

$$\text{score}_{10} = \frac{\text{raw_score}}{N} \quad (3)$$

The maximum possible value of `score10` is 10. In addition to this score, the experiments also track the answer extraction rate, score buckets, and performance by problem type.

D. Experimental Variants

Several experimental variants were implemented to study the effect of target format, preprocessing strategy, and inference policy. Among these variants, Variant 7 and Variant 11 are particularly important because they represent two different inference directions after answer-only fine-tuning.

Variant 7 uses answer-only LoRA fine-tuning combined with a rule-solver fallback during inference. In this variant, the prediction function first checks whether the input question matches a small set of conservative hand-crafted patterns, such as least common multiple, greatest common divisor, simple arithmetic expressions, triangle perimeter, and several direct arithmetic templates. If no rule is applicable, the model generates the answer using the fine-tuned GPT-2 model.

Therefore, Variant 7 should not be interpreted as a purely model-only system. Its validation score reflects a hybrid inference setting in which most predictions are still produced by the language model, but a small number of predictions are generated by deterministic rules.

Variant 11 removes the rule solver entirely and uses a model-only inference policy. In this variant, all candidate answers are generated by the fine-tuned model. The inference procedure first generates one greedy candidate and then optionally generates additional sampled candidates with temperature sampling. The final answer is selected by grouping equivalent candidate answers and choosing the most frequent canonical answer. This variant is useful for evaluating whether model-only voting can improve answer stability without relying on hand-crafted mathematical rules.

The final submitted configuration follows the model-only direction but simplifies the inference procedure further by using greedy decoding. This design avoids external solvers and hand-crafted answer generation, while keeping the prediction pipeline deterministic and efficient.

1) *Target Format Variants:* The first group of experiments compares different output targets. Full-solution targets require the model to generate a complete explanation before the final answer. Step-format targets convert the solution into ordered steps. Short-reasoning targets keep only a compact explanation. Answer-only targets train the model to generate only the final answer.

This comparison is important because the evaluation focuses on the final numerical answer rather than the quality of the explanation. Therefore, a shorter target may provide a stronger learning signal for the metric.

2) *Data Selection Variants:* The second group of experiments evaluates different training data selection strategies. Some variants keep all problem types and apply quality filtering. Other variants remove specific problem groups to test whether they introduce noise. In particular, several experiments analyze the effect of removing AnsAug samples while preserving Rephrased, SV, and FOFAR samples.

3) *Inference Variants:* The third group of experiments explores different decoding strategies, including greedy decoding, beam search, self-consistency voting, verifier-based candidate selection, and rule-based fallback. These variants are used to understand whether inference-time candidate selection can improve final answer quality.

4) *Final Model-only Variant:* Although some variants use rule-based components for analysis, the final selected configuration uses model-only inference. This means that the final answer is generated by the fine-tuned model without an additional solver or test-time adaptation. This choice keeps the final pipeline simple and aligned with the official task constraints.

E. Final Fine-tuning Configuration

The final model is fine-tuned using LoRA with an answer-only target. The training subset contains 40,000 selected examples. The GSM_AnsAug and MATH_AnsAug groups are

removed from the training subset, while the rephrased, solve-variable, and reverse-parameter groups are preserved.

Table IX summarizes the final fine-tuning configuration.

Table IX
Final fine-tuning configuration

Component	Value
Base model	NlpHUST/gpt2-vietnamese
Fine-tuning method	LoRA
Target format	Answer-only
Training samples	40,000
Maximum sequence length	256
Maximum new tokens	12
Epochs	5
Learning rate	5×10^{-4}
LoRA rank	32
LoRA alpha	64
LoRA dropout	0.03
Decoding strategy	Greedy decoding
Rule solver	Not used

The final training data distribution is shown in Table X. The selected distribution reflects the decision to remove answer-augmented samples while preserving problem types that require objective transformation, such as solve-variable and reverse-parameter problems.

Table X
Training distribution of the final configuration

Problem type	Selected samples
GSM_Rephrased	14,476
GSM_SV	7,344
GSM_FOVAR	6,268
MATH_Rephrased	8,012
MATH_SV	1,942
MATH_FOVAR	1,958
Total	40,000

F. Validation and Output Files

During Phase 1, the validation set is used only for development and error analysis. The notebook generates validation outputs and reports to compare variants and inspect model errors. The expected validation artifacts include:

- `valid_output.json`: generated predictions on the validation set;
- `valid_report.json`: validation score, score buckets, type-level results, and error statistics.

During the official test phase, the notebook must generate the file:

`test_predictions.json`

The file is a JSON array where each element contains the example id, input question, problem type, and model output:

```
[
  {
    "id": 0,
    "query_vi": "Question text",
    "type": "GSM_Rephrased",
    "model_output": "Đáp án là: ..."
```

```
}
]
```

This output format ensures that the final prediction can be automatically evaluated by extracting the numerical answer from the `model_output` field.

V. Results and Discussion

This section presents the validation results obtained from the experimental variants. The discussion focuses on how different target formats, data selection strategies, and inference designs affect the final numerical answer score. Only variants with reliable saved validation outputs are included in the main comparison. Variants whose notebook outputs were stale or incomplete are excluded from the quantitative table and are discussed only qualitatively when relevant.

A. Overall Validation Results

Table XI summarizes the main experimental variants. The variant numbers follow the chronological notebook experiments, while the final row represents the selected configuration used for the final submission. The reported score is `score_10`, where the maximum possible value is 10. The results should be interpreted as a comparison of complete experimental pipelines rather than perfectly controlled single-factor ablations, because some variants differ in both training data selection and inference strategy.

The results indicate several important trends. First, variants using answer-only targets generally outperform variants that require the model to generate reasoning text before the answer. Second, general data filtering alone is not sufficient; the distribution of problem types has a strong effect on validation performance. Third, although rule-based fallback can help in some specific cases, the best result among the tested variants is achieved by a model-only configuration. This suggests that, within our experimental setting, target simplification and data selection have a larger impact than increasing inference-time complexity.

B. Effect of Target Format

The initial baseline and several early variants used solution-style targets, where the model generated a full or partial reasoning process before producing the final answer. This design is natural for mathematical word problems, but it is not necessarily optimal for the scoring metric used in this task. Since the evaluation only depends on the final numerical answer, long reasoning outputs introduce additional failure points, including arithmetic mistakes, repetition, formatting errors, and truncation before the final answer.

The short-reasoning variant achieved a validation `score_10` of 0.721. Although it reduced the output length compared with full-solution generation, the model still needed to generate intermediate text before producing the final answer. In many cases, the generated reasoning contained arithmetic mistakes or irrelevant explanations, which eventually led to an incorrect final answer.

Table XI
Overall validation results of main experimental variants

Variant	Main idea	Inference policy	Rule solver	score_10
Variant 5	Answer-only training using only direct problem types	Greedy decoding	No	0.323
Variant 6	Short-reasoning target with verifier-based candidate selection	Verifier selection	No	0.721
Variant 7	Answer-only LoRA with rule-solver fallback	Rule-first + model generation	Yes	1.578
Variant 11	Answer-only LoRA with model-only voting	Greedy + sampled candidates + voting	No	1.403
Variant 13	Quality-filtered 40k answer-only training with greedy decoding	Greedy decoding	No	1.373
Variant 15	Answer-only training after dropping multiple problem types	Greedy decoding	No	1.540
Final	Answer-only training, drop only AnsAug, model-only greedy decoding	Greedy decoding	No	1.995

In contrast, answer-only variants achieved higher scores. By training the model to generate only the final answer, the learning objective becomes more directly aligned with the evaluation metric. This target format also reduces output instability, repeated text, and answer extraction failures.

Table XII
Effect of target format on validation performance

Variant	Target format	score_10
Variant 6	Short reasoning	0.721
Variant 11	Answer-only	1.403
Variant 13	Answer-only	1.373
Final	Answer-only	1.995

This comparison suggests that target simplification is one of the most important design choices in this task. Chain-of-Thought prompting has been shown to be effective for sufficiently large language models [3], but the Vietnamese GPT-2 model used in this work is much smaller and is not specialized for mathematical reasoning. In our experiments, directly learning the final answer was more effective than forcing the model to generate long reasoning traces.

C. Effect of Data Selection

Data selection has a significant influence on the final result. Variant 5 used answer-only training but kept only direct problem types. This produced a low score_10 of 0.323 because the model was not exposed to solve-variable and reverse-parameter problem types during training. This result indicates that the SV and FOBAR groups should not be removed entirely.

Variant 15 improved the score to 1.540 by dropping several noisy groups. However, this strategy also removed some useful hard problem types, including parts of the FOBAR and SV groups. As a result, the model lost coverage for some non-direct objectives.

The final configuration uses a more selective strategy. It removes only the two answer-augmented groups, GSM_AnsAug and MATH_AnsAug, while preserving Rephrased, SV, and FOBAR samples. This strategy achieves the best validation score among the tested variants. However, this should be interpreted as an empirical result under our validation setting, not as a general claim that all answer-augmented examples are harmful.

Table XIII
Effect of data selection strategy

Variant	Data selection strategy	score_10
Variant 5	Keep only direct problem types	0.323
Variant 13	Quality-filtered 40k samples, all types retained	1.373
Variant 15	Drop multiple groups, including some hard types	1.540
Final	Drop only GSM_AnsAug and MATH_AnsAug	1.995

This result highlights that data filtering should not only remove noisy samples but also preserve important problem objectives. In particular, retaining solve-variable and reverse-parameter examples is important for generalization across problem types. These findings are consistent with recent work showing that high-quality data selection can strongly affect the performance of small language models on mathematical reasoning tasks [5].

D. Effect of Inference Strategy

Several inference strategies were tested, including greedy decoding, beam search, model-only voting, verifier-based selection, and rule-based fallback. The experiments show that more complex inference procedures do not necessarily lead to better validation performance.

Variant 7 used answer-only fine-tuning with a rule-solver fallback and achieved a score_10 of 1.578. In this variant, a small set of hand-crafted rules was applied before model generation. If the question matched a conservative rule pattern, the rule solver produced the answer; otherwise, the fine-tuned model generated the prediction. Therefore, Variant 7 should be interpreted as a hybrid rule-first pipeline rather than a purely model-only system. Its result shows that deterministic rules can help in some specific patterns, but they also make the pipeline less representative of the language model’s own capability.

Variant 11 removed the rule solver entirely and used model-only voting. It generated one greedy candidate and additional sampled candidates, then selected the most frequent canonical answer. This setting is cleaner because all predictions come from the fine-tuned model. However, Variant 11 achieved a lower score_10 of 1.403. This suggests that voting alone cannot compensate for the model’s weak numerical reasoning. If the same model repeatedly generates similar incorrect answers, majority voting may reinforce the error rather than correct it.

The final configuration uses model-only greedy decoding and achieves the best validation score among the tested variants. Compared with Variant 7, it does not rely on hand-crafted answer generation. Compared with Variant 11, it avoids the extra cost and instability of sampled candidate generation. The main improvement comes from the training target and data selection strategy rather than from inference-time complexity.

Table XIV
Comparison of inference strategies

Variant	Inference strategy	score_10
Variant 7	Rule-first fallback + model generation	1.578
Variant 11	Model-only candidate voting	1.403
Final	Model-only greedy decoding	1.995

The final configuration therefore uses greedy decoding. This choice is simple, deterministic, efficient, and compatible with the constraint that the model must remain frozen during inference.

E. Final Model Performance

The final model achieves a validation score_10 of 1.995 with an answer extraction rate of 1.0. The score distribution is shown in Table XV. Out of 997 validation examples, 151 examples receive the maximum score of 10, while 527 examples receive 0. Although this is the best result among the tested variants, the absolute performance remains limited because more than half of the validation examples are still incorrect or too inaccurate.

Table XV
Score bucket distribution of the final model

Per-example score	Number of examples
10	151
5	40
1	279
0	527
Total	997

The final model also achieves a complete answer extraction rate. This means that every validation output contains a parseable answer. This is an important improvement over generation settings that produce long or unstable reasoning text, where answer extraction can fail. However, a parseable answer is not necessarily a correct answer. The remaining bottleneck is therefore numerical correctness rather than output formatting alone.

F. Performance by Problem Type

Table XVI reports the validation performance of the final model by problem type. The best-performing groups are MATH_FOVAR, GSM_Rephrased, and GSM_FOVAR. The weakest group is GSM_AnsAug.

Table XVI
Final model validation performance by problem type

Problem type	Examples	score_10
GSM_AnsAug	209	1.249
GSM_SV	97	1.526
MATH_AnsAug	171	1.889
MATH_Rephrased	115	2.017
MATH_SV	41	2.122
GSM_FOVAR	122	2.320
GSM_Rephrased	197	2.690
MATH_FOVAR	45	2.778

An interesting observation is that the final model performs relatively well on MATH_AnsAug, even though MATH_AnsAug is removed from the final training subset. This suggests that the model can transfer from MATH_Rephrased, MATH_SV, and MATH_FOVAR to some answer-augmented MATH examples. In contrast, GSM_AnsAug remains the weakest group, indicating that removing all GSM answer-augmented samples may reduce coverage for some everyday arithmetic patterns.

The strong performance on FOVAR groups also supports the decision to preserve reverse-parameter examples during training. Earlier variants that removed hard problem types had weaker coverage on non-direct problem objectives.

G. Ablation Findings

The experimental results lead to several findings.

1) *Answer-only Training is More Effective than Reasoning Generation*: The variants that require the model to generate reasoning text generally perform worse than answer-only variants. This is because the score depends only on the final numerical answer, while reasoning generation increases the probability of arithmetic mistakes, repetition, formatting errors, and answer truncation.

2) *Problem-type Selection is More Important than General Quality Filtering*: Variant 13 shows that selecting 40,000 generally clean samples is not enough to obtain the best score. The final model performs better because it changes the problem-type distribution by removing answer-augmented groups while retaining rephrased, solve-variable, and reverse-parameter problems.

3) *Solve-variable and Reverse-parameter Samples Should be Preserved*: Dropping hard types can reduce coverage and harm type-level performance. The final model keeps SV and FOVAR samples, which helps it perform better on non-direct problem objectives.

4) *Rule-based Components are Helpful but Not Required for the Best Tested Result*: Rule-based fallback improves some cases in Variant 7, but the best validation score among the tested variants is achieved without rule-based answer generation. This does not mean that rule-based methods are generally ineffective. Rather, it suggests that in our setting, a cleaner model-only pipeline with better target design and data selection performs better than adding a small hand-crafted fallback module.

5) *Model-only Voting Does Not Guarantee Better Numerical Accuracy*: Variant 11 shows that generating multiple candidates from the same model does not necessarily improve

the final score. Voting can improve stability when the model produces diverse candidates around the correct answer. However, when the model has a systematic numerical bias, the sampled candidates may contain repeated wrong answers. In that case, voting may select a frequent but incorrect prediction.

H. Discussion

Overall, the experiments show that the main bottleneck is not only the decoding strategy but also the alignment between the training objective, the data distribution, and the evaluation metric. A model trained to generate long solutions may produce fluent text but still fail to output the correct number. Conversely, an answer-only model with carefully selected training data can produce more stable and extractable outputs.

However, the final validation score also shows that the model has not acquired robust mathematical reasoning ability. More than half of the validation examples still receive zero score, even though the extraction rate is 1.0. Therefore, the improvement should be understood as better answer stability and better alignment with the metric, not as a complete solution to mathematical reasoning.

Within the tested configurations, simplifying the target and controlling the training distribution are more effective than increasing inference complexity. This conclusion is especially relevant under the runtime and policy constraints of the Kaggle setting, where the notebook must run within a limited time and cannot rely on external solvers, online resources, or test-time learning.

VI. Error Analysis, Limitations, and Future Work

This section discusses the remaining weaknesses of the final model, the limitations of the current experimental setting, and several possible directions for future improvement. Although the final configuration achieves the best validation score among the tested variants, the absolute performance remains limited. The main remaining challenges are numerical reasoning errors, uneven performance across problem types, and ambiguity in answer parsing.

A. Error Analysis

The final model achieves an answer extraction rate of 1.0, meaning that every validation output contains a parseable answer. However, this does not imply that the answers are numerically correct. Many outputs are well formatted but still receive zero score because the predicted value is far from the gold answer. Therefore, the main bottleneck of the final model is not answer extraction, but numerical correctness.

The error patterns observed in Variant 7 and Variant 11 further support this conclusion. Variant 7 uses a hybrid rule-first pipeline, while Variant 11 uses model-only voting. Both variants can produce extractable answers, but they still suffer from numerical errors. This indicates that improving output format alone is insufficient; the model must also improve its ability to understand the problem objective and perform the required arithmetic operation.

1) *Large Numerical Hallucination*: One of the most common error patterns is large numerical hallucination. In several validation cases, the gold answer is a small number such as 2, 3, 10, or 20, while the model predicts much larger values such as 120, 325, 720, 3400, or 4000. These errors suggest that the model sometimes learns surface-level numerical patterns from the training data instead of performing reliable arithmetic reasoning.

This issue is especially important because the evaluation metric is based on relative error. When the generated answer is far from the gold answer, the example receives zero score even if the output format is valid. Therefore, producing a parseable answer is not sufficient; the predicted answer must also be numerically reasonable.

2) *Weakness on GSM_AnsAug*: The final model performs weakest on the GSM_AnsAug group. This is consistent with the final data selection strategy, where both GSM_AnsAug and MATH_AnsAug are removed from the training subset. Although this strategy improves the overall validation score, it reduces direct exposure to some answer-augmented GSM patterns.

This result shows a trade-off in data selection. Removing noisy augmented data can improve generalization, but removing all samples from a group may also reduce coverage for some problem styles. In future work, a small number of high-quality GSM_AnsAug samples could be reintroduced to improve coverage without reintroducing too much noise.

3) *Arithmetic Inconsistency*: Since the final model uses answer-only generation, it does not explicitly produce intermediate reasoning steps. This makes the output shorter and more stable, but it also means that there is no explicit calculation trace to inspect or verify. When the model predicts an incorrect number, it is difficult to determine whether the error comes from misunderstanding the question, choosing the wrong operation, or performing incorrect arithmetic.

Earlier variants that generated short reasoning or full reasoning provided more interpretable outputs, but their numerical performance was weaker. This reveals a trade-off between interpretability and final-answer accuracy. In this task setting, answer-only generation improves alignment with the scoring metric, but it does not solve the underlying reasoning problem.

4) *Limitations of Model-only Voting*: Variant 11 shows that model-only voting does not necessarily improve numerical accuracy. The voting strategy generates multiple candidates from the same fine-tuned model and selects the most frequent canonical answer. However, if the model has a systematic numerical bias, the sampled candidates may contain repeated or similar wrong answers.

In such cases, voting improves output stability but does not introduce new mathematical knowledge. Therefore, majority voting may select a frequent but incorrect answer. This explains why model-only voting did not outperform the final greedy configuration in our experiments.

5) *Answer Parsing Ambiguity*: The scoring process depends on extracting a numerical answer from the generated text. Although the final model achieves an extraction rate of 1.0 on validation, answer parsing remains a potential source

of error. In particular, decimal separators and thousands separators can be ambiguous. For example, a number such as 40.320 may be interpreted either as a decimal value or as a thousands-separated integer depending on the parser design.

This issue is more likely to affect MATH-style problems, where answers may contain fractions, decimals, radicals, or LaTeX expressions. A more robust parser could reduce evaluation errors caused by formatting ambiguity.

B. Limitations

1) *Limited Mathematical Reasoning Ability*: The base model used in this work is a Vietnamese GPT-2 model. Although it can learn language patterns and answer formats through fine-tuning, it is not a specialized symbolic or arithmetic solver. As a result, it may still fail on problems requiring multi-step reasoning, exact calculation, algebraic manipulation, or careful interpretation of constraints.

The answer-only objective improves score alignment, but it does not explicitly teach the model to reason through a problem. Therefore, the final model should be viewed as a fine-tuned generative answer predictor rather than a reliable mathematical reasoning system.

2) *Dependence on Validation Distribution*: The final configuration is selected based on validation performance. Since the hidden test distribution may differ from the validation set, the chosen data filtering strategy may not always generalize perfectly. In particular, the decision to remove all AnsAug samples is beneficial on the validation set, but it may be less optimal if the official test set contains many answer-augmented patterns.

This limitation is common in empirical model selection. To reduce this risk, future experiments could use multiple validation splits or grouped cross-validation based on original problem identifiers. This would make the model selection process less dependent on a single validation distribution.

3) *Incomplete Controlled Ablation*: The reported variants are useful for understanding experimental trends, but they are not perfectly controlled single-factor ablations. Some variants differ in more than one aspect, such as target format, data filtering, inference strategy, and candidate selection. Therefore, the comparison should be interpreted as a comparison of complete experimental pipelines rather than a strict isolation of each individual factor.

A more rigorous ablation study would keep all settings fixed and change only one factor at a time. For example, one could compare greedy decoding and voting under the exact same training subset, checkpoint, and prompt template.

4) *Filtering of Long and Complex Samples*: The preprocessing pipeline removes or filters some long, diagram-heavy, or heavily LaTeX-based samples. This improves training stability and reduces truncation risk, but it may also reduce the model’s ability to handle complex MATH problems. In particular, geometry problems, symbolic expressions, and problems with diagrams may be underrepresented in the final training subset.

Thus, the final model is optimized for stable numerical answer generation under the observed validation distribution, but it may still struggle with long-form mathematical inputs.

5) *No External Solver or Test-time Adaptation*: The final system intentionally avoids rule-based solvers, external tools, online resources, and test-time learning. This design keeps the method simple and compliant with the task constraints, but it also limits the ability to correct obvious arithmetic errors during inference.

Variant 7 shows that a small rule-based fallback can improve some specific cases. However, the final method does not use rule-based answer generation in order to keep the inference pipeline model-only and more representative of the fine-tuned language model itself.

C. Future Work

1) *Selective Reintroduction of High-quality AnsAug Samples*: The final model removes both GSM_AnsAug and MATH_AnsAug. While this improves the overall validation score, the weak performance on GSM_AnsAug suggests that completely removing this group may be too aggressive. A possible improvement is to reintroduce a small subset of high-quality AnsAug samples selected by strict criteria, such as clear answer anchors, short query length, non-conflicting answers, and low formatting noise.

2) *Checkpoint Selection by score₁₀*: The final model is selected from the last training checkpoint. However, the lowest validation loss may occur before the final step, and validation loss is not perfectly aligned with score₁₀. Future experiments should evaluate multiple checkpoints directly using score₁₀ and select the best checkpoint based on the actual task metric rather than loss alone.

3) *Improved Numerical Parsing*: A more robust answer parser could reduce errors caused by decimal notation, thousands separators, fractions, and LaTeX expressions. This would improve both validation analysis and final prediction extraction. The parser should distinguish between decimal dots and thousands separators using context, and it should handle common mathematical forms such as fractions and boxed answers more consistently.

4) *Lightweight Model-only Verification*: Although the final system avoids rule-based solving, a lightweight model-only verification strategy could still be explored. For example, the model could generate several answer candidates, and a separate model-based selection mechanism could choose the most consistent candidate without using external solvers. This approach may preserve compliance with the model-only setting while reducing random generation errors.

However, this direction should be evaluated carefully because Variant 11 shows that simple majority voting is not sufficient. A useful verifier would need to evaluate candidate consistency more effectively than frequency-based voting.

5) *Answer-only Curriculum Learning*: Curriculum learning was explored in earlier variants mainly with solution-style targets. A future direction is to apply curriculum learning under the answer-only setting. The model could first learn

direct and rephrased problems, then gradually incorporate solve-variable and reverse-parameter problems. This may help the model learn simpler numerical mappings before handling objective-changing problem types.

6) *More Controlled Ablation Experiments*: Future work should also include more controlled ablation experiments. For example, the same training subset could be used to compare greedy decoding, model-only voting, and rule-based fallback. Similarly, the effect of removing AnsAug samples could be tested while keeping the number of training samples, learning rate, epoch count, and decoding strategy fixed. This would make it easier to identify which design choice contributes most to the final score.

VII. Conclusion

This report presented an experimental study on fine-tuning a Vietnamese GPT-2 model for Vietnamese mathematical word problem answering. Under the fixed-model and limited-runtime constraints of the task, the experiments show that target design and data selection have a stronger observed impact on validation performance than increasing the complexity of decoding or post-processing. In particular, solution-style generation often leads to unstable reasoning paths, arithmetic mistakes, repeated text, and formatting errors, while answer-only fine-tuning aligns the model output more directly with the final-answer-based evaluation metric.

The comparison between the main experimental variants also highlights the trade-off between inference complexity and model simplicity. Variant 7 demonstrates that a small rule-based fallback can improve some specific cases, but it makes the system a hybrid rule-first pipeline rather than a purely model-based approach. Variant 11 removes rule-based components and uses model-only voting, but the results show that voting cannot reliably correct numerical errors when all candidates are generated by the same small language model. Therefore, the final configuration adopts a simpler model-only greedy decoding strategy and focuses improvement on the training objective and data distribution.

The final configuration uses LoRA fine-tuning with an answer-only target, removes the answer-augmented groups, preserves rephrased, solve-variable, and reverse-parameter samples, and applies model-only greedy decoding. This configuration achieved the best validation result among the tested variants, with a `score_10` of 1.995 and an answer extraction rate of 1.0.

However, the result should be interpreted carefully. The complete extraction rate shows that the model can produce parseable answers, but it does not imply robust mathematical reasoning. More than half of the validation examples still receive zero score, and the model can generate numerically implausible answers. Therefore, the main contribution of this work is not to fully solve mathematical reasoning with Vietnamese GPT-2, but to show that, under constrained small-model settings, careful target simplification and data selection can improve answer stability and validation performance.

Code Availability

The implementation of this project is publicly available in the `viet-gpt2-math-word-problem` Github repository. The repository includes the data preprocessing pipeline, LoRA fine-tuning notebooks, validation scripts, experiment variants, and the final model-only prediction pipeline used to generate the reported validation results.

References

- [1] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano *et al.*, “Training verifiers to solve math word problems,” *arXiv preprint arXiv:2110.14168*, 2021.
- [2] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” in *International Conference on Learning Representations*, 2022.
- [3] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24 824–24 837.
- [4] J. Uesato, N. Kushman, R. Kumar, F. Song, N. Siegel, L. McLean, Y. Wu, and G. Irving, “Solving math word problems with process-and outcome-based feedback,” *arXiv preprint arXiv:2211.14275*, 2022.
- [5] B. Liu, S. Bubeck, R. Eldan, J. Kulkarni, Y. Li, R. Ward, and J. Zhang, “Tinygsm: achieving > 80% on gsm8k with small language models,” *arXiv preprint arXiv:2312.09241*, 2023.